Simple Encryption (FlagYard) – WriteUp

Debonmoy Pal

Name – Simple Encryption

Desc - Just a simple encryption algorithm based on basic arithmetic operations.

Code Analysis

```python
from secrets import randbelow

M = 2**128

def enc(msg, key):
    o = [2, 73]
    for i, p in enumerate(map(ord, msg)):
        o.append(((key*o[i+1])^(key+(o[i]*p))) % M)
    return o

key = randbelow(2**64)
flag = open('flag.txt', 'r').read().strip()
ct = enc(flag, key)
print(ct)
```

The encryption uses a modulus of 2^128 which means the result of each step is reduced modulo 2^128.

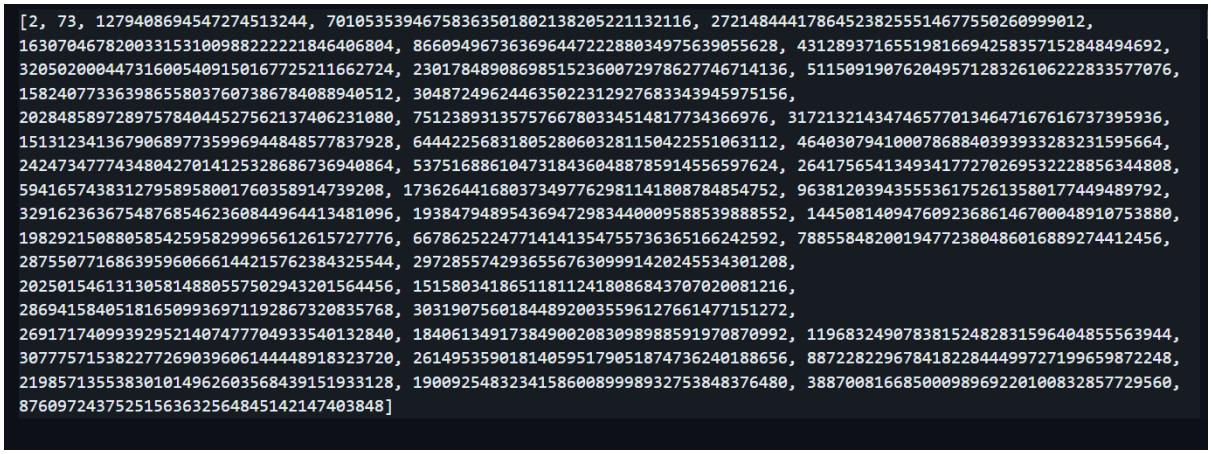
The algorithm starts with 2 known values 2 and 73.

A random key is generated between 0 and (2^64 -1). I don't know the key, but the size is limited to 64 bits.

The main encryption formula: o.append(((key*o[i+1])^(key+(o[i]*p))) % M)

Here p is the numeric value of a plain text character.

Output Analysis:


[2, 73, 1279408694547274513244, 701053539467583635018021382052211321 16, 272148444178645238255514677550260999012,
1630704678200331531009882222218464068 04, 866094967363696447222880349756390556 28, 431289371655198166942585357152848494692,
3205020004473160054091501677252116627 24, 230178489086985152360072978627746714136, 511509190762049571283261062228333577076,
1582407733639865580376073867840889405 12, 304872496244635022312927683343945975156,
2028485897289757840445275621374062310 80, 751238931357576678033451481773436697 6, 317213214347465770134647167616737395936,
1513123413679068977359969448485778379 28, 644422568318052806032811504225510631 12, 464030794100078688403939332832315956 64,
2424734777434804270141253286867369408 64, 537516886104731843604887859145565976 24, 264175654134934177270269532228856344808,
5941657438312795895800176035891473920 8, 173626441680373497762981141808784854752, 963812039435553617526135801774494897 92,
3291623636754876854623608449644134810 96, 193847948954369472983440009588539888552, 144508140947609236861467000489107538 80,
1982921508805854259582999656126157277 76, 667862522477141413547557363651662425 92, 788558482001947723804860168892744124 56,
2875507716863959606661442157623843255 44, 297285574293655676309991420245534301208,
2025015461313058148805575029432015644 56, 151580341865118112418086843707020081216,
2869415840518165099369711928673208357 68, 303190756018448920035596127661477151272,
2691717409939295214074777049335401328 40, 184061349173849002083098898591970870992, 119683249078381524828315964048555639 44,
3077757153822772690396061444489183237 20, 261495359018140595179051874736240188656, 887228229678418228444997271996598722 48,
2198571355383010149626035684391519331 28, 190092548323415860089998932753848376480, 388700816685000989692201008328577295 60,
8760972437525156363256484514214740384 8]

The file contains a list of large integers where the first 2 numbers are not encrypted characters and the remaining numbers are generated during encryption. There are 47 integers in total where 45 are encrypted characters. I now need to reverse the encryption process.

Vulnerability Identification:

The main weakness is the relationship between consecutive ciphertext values.

Initial values are known to me and I get the subsequent values from the ciphertext.

I also know the key is smaller than 2^64. These clues provide useful mathematical snippets for recovering the key.

Once the key is known, I can test each character against the encryption formula to recover the plaintext.

Key recovery:

From the source code, I know:

M = 2**128

o = [2, 73]

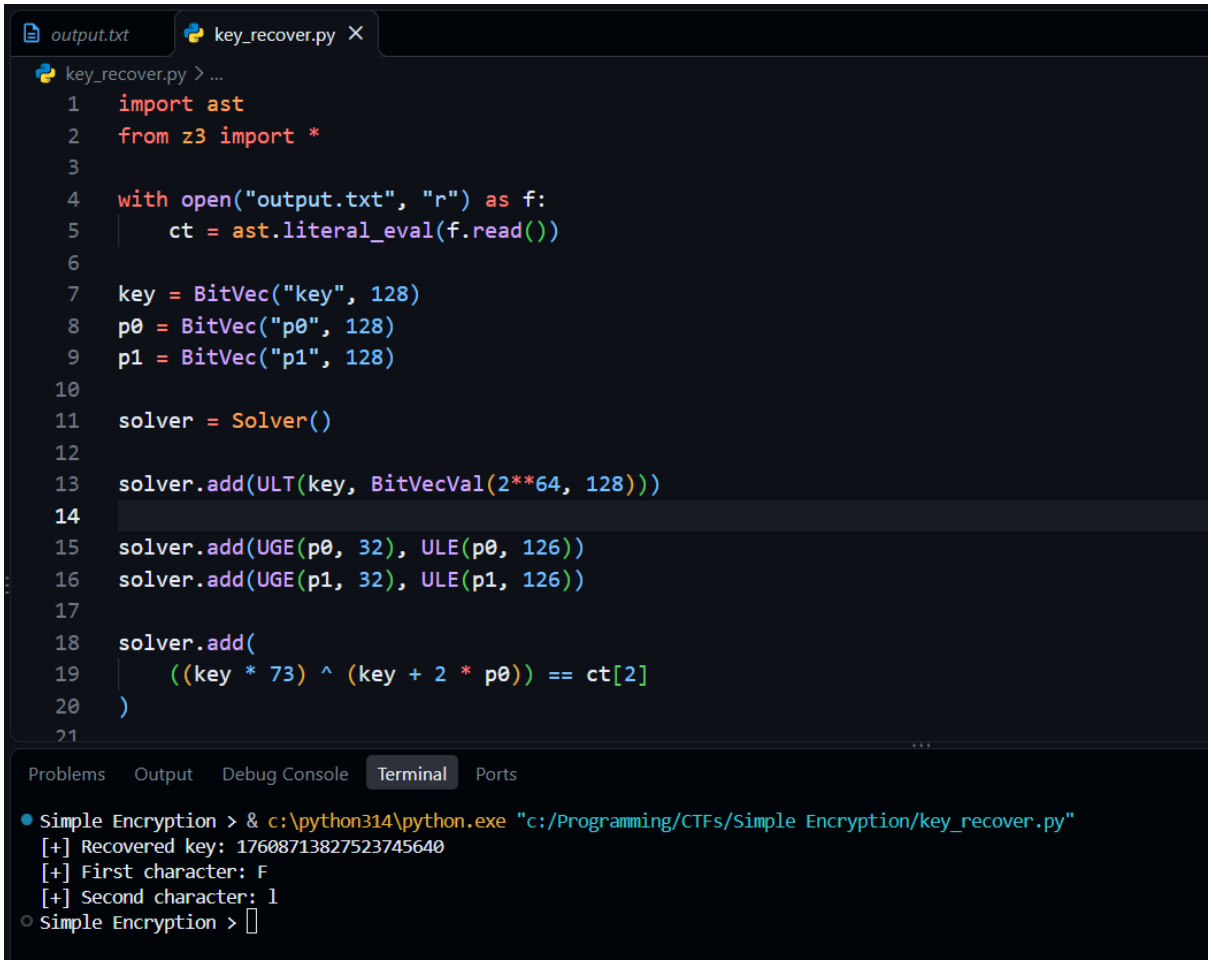
key = randbelow(2**64)

And the first encrypted value:
c2 = ((key * 73) ^ (key + 2 * p0)) % M
I know the value of c2 from output.txt but both key and the 1st plaintext character p0 are unknown.

If there's a possibility of 64-bit integer key and the first character is a printable ascii then:

Only 95 possible printable ascii characters exists from decimal 32 to 126.

So we use an SMT Solver to recover the key:



The key recovered is: 17608713827523745640

Flag recovery:

Now to find the flag, for each position, I try printable ascii character and check which one reproduces the corresponding ciphertext value.
The original formula:

((key * previous_value) ^ (key + previous_previous_value * ord(character))) % M

For every candidate character:

1. Convert the character to its numeric ASCII value.
2. Apply the encryption formula using the previous two values.

3. Compare the result with the corresponding ciphertext value.
4. Keep the character if the result matches.

And I have to repeat this process for all 45 characters.



Final Flag:
FlagY{e4sy_3nc_3asy_d3c_a6cebdf01bf8a8feb61f}